

🎯 50 технических вопросов и ответов для собеседования по C#/.NET

Язык C# и ООП

1. Каковы основные принципы ООП в C#?

Инкапсуляция, наследование, полиморфизм и абстракция.

2. В чем разница между классом и структурой?

Классы — это ссылочные типы; структуры — это типы значений, хранящиеся в стеке.

3. В чем разница между ключевыми словами `override`, `virtual` и `new`?

- `virtual`: позволяет переопределять методы
- `override`: переопределяет виртуальный метод
- `new`: скрывает метод базового класса

4. Что такое внедрение зависимостей?

Шаблон, при котором зависимости предоставляются извне, а не создаются внутри класса.

5. В чем разница между `Task` и `ValueTask`?

`ValueTask` позволяет избежать выделения памяти для часто выполняемых задач.

Среда выполнения и память .NET

6. Что такое CLR?

Среда выполнения, которая управляет памятью, JIT-компиляцией и сборкой мусора.

7. Что такое сборка мусора в .NET?

Автоматическое управление памятью, освобождающее неиспользуемые объекты.

8. Что такое поколения в сборке мусора?

Gen0, Gen1, Gen2 — оптимизация очистки памяти на основе времени жизни объекта.

9. В чем разница между стеком и кучей?

Стек хранит типы значений и фреймы функций; куча хранит ссылочные типы.

10. Что такое упаковка/распаковка?

Преобразование типов значений в объекты (упаковка) и обратно (распаковка).

ASP.NET / Web API

11. Что такое промежуточное ПО в ASP.NET Core?

Компоненты, обрабатывающие запросы в конвейере.

12. В чем разница между Web API и MVC?

MVC возвращает представления; Web API возвращает данные (JSON/XML).

13. Как включить CORS?

С помощью AddCors() и UseCors() в конвейере.

14. Что такое привязка модели?

Автоматическое сопоставление данных запроса с параметрами метода.

15. Как защитить API?

JWT, OAuth2, HTTPS, валидация, ограничение скорости запросов.

Entity Framework / Базы данных

16. В чем разница между немедленной, отложенной и явной загрузкой?

- Немедленная загрузка: загружает связанные данные немедленно
- Отложенная загрузка: загружает при обращении
- Явная загрузка: запускается вручную

17. Что такое миграция?

Версионированное изменение схемы базы данных.

18. Как оптимизировать производительность EF?

Используйте AsNoTracking, проекции, индексы и избегайте запросов N+1.

19. Что такое шаблон репозитория?

Слой, который абстрагирует логику доступа к данным.

20. Что такое LINQ?

Язык запросов, интегрированный в C#.

Архитектура (DDD, CQRS, MediaTek, микросервисы)

21. Что такое DDD?

Методология, ориентированная на моделирование реальных бизнес-доменов.

22. Что такое CQRS?

Разделение моделей чтения и записи для масштабируемости.

23. Для чего используется MediaTek?

Разделение обработки запросов с помощью шаблона «посредник».

24. Что такое корневой агрегат?

Основная сущность, контролирующая доступ к связанным объектам.

25. Каковы преимущества микросервисов?

Масштабируемость, независимое развертывание, изоляция ошибок.

Взаимодействие фронтенда (Angular, React, TypeScript)

26. В чем разница между Angular и React?

Angular — это полноценный фреймворк; React — это библиотека пользовательского интерфейса.

27. Для чего используется Redux?

Централизованное управление состоянием.

28. Что такое MVVM?

Шаблон пользовательского интерфейса, разделяющий View, ViewModel и Model.

29. Как оптимизировать производительность фронтенда?

Ленивая загрузка, мемоизация, разделение кода.

30. Что такое TypeScript?

Типифицированное расширение JavaScript.

Системы реального времени / Уведомления

31. Как реализовать связь в реальном времени?

SignalR, WebSockets.

32. Как работает интеграция с Twilio?

Использование REST API Twilio для отправки SMS/голосовых сообщений.

33. Для чего используется SMTP?

Для отправки электронных писем.

34. Как спроектировать чат-сервер?

WebSockets, очереди сообщений, постоянное хранилище.

35. Что такое FFmpeg?

Инструмент для обработки видео (кодирование, обрезка, анализ).

Платежи / Интеграции

36. Как работает интеграция с API Stripe?

Создание платежных намерений, обработка веб-хуков, обработка транзакций.

37. Что такое веб-хук?

URL-адрес обратного вызова для получения внешних событий.

38. Что такое GraphQL?

Язык запросов, позволяющий клиентам запрашивать определенные данные.

39. Как безопасно интегрировать API сторонних разработчиков?

Повторные попытки, кэширование, ограничение скорости запросов, обработка ошибок.

40. Для чего используется Firebase?

Аутентификация, база данных реального времени, облачный обмен сообщениями.

Модульное тестирование / CI/CD

41. В чем разница между xUnit и NUnit?

Оба являются тестовыми фреймворками; xUnit более современный.

42. Что такое мокирование?

Имитация зависимостей для изоляции логики.

43. Что такое CI/CD?

Автоматизированная сборка, тестирование и развертывание.

44. Что такое Swashbuckle?

Генерирует документацию Swagger для API.

45. Что делает модульный тест хорошим?

Изолированный, детерминированный, читаемый и быстрый.

Производительность и масштабируемость

46. Как оптимизировать производительность .NET?

Кэширование, асинхронное программирование, профилирование, минимизация выделения памяти.

47. Что такое кэширование?

Хранение часто используемых данных для более быстрого доступа.

48. Как масштабировать .NET-приложение?

Балансировка нагрузки, микросервисы, горизонтальное масштабирование.

49. Как обрабатывать большие объемы XML-данных?

Потоковая передача, индексирование, оптимизированное хранилище SQL.

50. Что такое YSlow?

Инструмент для анализа производительности фронтенда.

Вопросы и ответы по культуре и поведению (адаптировано к резюме Руслана Мелушева)

1. Каково было ваше самое большое достижение в вашем первом .NET-проекте?

Одним из моих первых достижений было создание полноценного веб-приложения с использованием Angular и Angular. 1. SP.NET с использованием принципов чистой архитектуры, таких как паттерн репозитория, CQRS и DDD. Это был мой первый опыт реализации чат-сервера в реальном времени с использованием MVVM, и я успешно интегрировал Twilio и SMTP-уведомления. Этот проект помог мне понять, как проектировать масштабируемые системы, и придал мне уверенности в выполнении более сложных задач на бэкенде.

2. Расскажите о случае, когда вы улучшили производительность в проекте.

В проекте Российской Премьер-лиги я оптимизировал обработку больших XML-наборов данных, реструктурировав SQL-запросы и проиндексировав базу данных. Это значительно сократило время обработки данных и улучшило скорость отклика платформы анализа матчей. Улучшение позволило аналитикам получать доступ к информации в реальном времени без задержек.

3. Опишите ситуацию, когда вам приходилось наставлять или поддерживать младших разработчиков.

В Drom и DARS я регулярно наставлял младших разработчиков, проверяя их код, объясняя архитектурные шаблоны и помогая им внедрять безопасные методы кодирования. Я также познакомил их с модульным тестированием с помощью xUnit и NUnit. Со временем это повысило производительность команды и сократило количество регрессий в производственной среде.

4. Как вы работаете в распределенных или удаленных командах?

Я работал в распределенных командах в нескольких компаниях, поэтому коммуникация и прозрачность имеют важное значение. Я полагаюсь на четкую документацию, регулярные обновления и такие инструменты, как Azure DevOps, GitHub и Trello. Я также стараюсь

заблаговременно уточнять требования и согласовывать ожидания, чтобы избежать недоразумений.

5. Что вас мотивирует как разработчика?

Меня мотивирует решение сложных технических задач и создание систем, которыми люди действительно пользуются. Мне нравится проектировать чистые архитектуры, улучшать производительность и видеть измеримый эффект — будь то более быстрая загрузка, улучшенный пользовательский опыт или более стабильное развертывание. Мне также нравится изучать новые технологии и применять их в реальных проектах.

6. Расскажите о случае, когда вам пришлось быстро освоить новую технологию.

Когда я присоединился к проекту Российской Премьер-лиги, мне пришлось работать с FFmpeg для расширенной обработки видео. Я быстро освоил интеграцию с ASP.NET Web Forms и автоматизацию рабочих процессов анализа видео. Это позволило нам создать платформу, способную эффективно обрабатывать видеоматериалы матчей.

7. Опишите конфликт или разногласие, возникшее в вашей команде, и как вы его разрешили.

В одном проекте возникли разногласия по поводу того, следует ли использовать монолитную или микросервисную архитектуру. Я предложил гибридный подход: разделить только наиболее важные модули на микросервисы, оставив остальные в монолите. Я подкрепил это данными о производительности и соображениями по обслуживанию. Команда согласилась, и решение улучшило масштабируемость без излишнего усложнения системы.

8. Какое ваше самое большое техническое достижение?

Одним из моих самых больших достижений стало проектирование трехзвенного микросервисного бэкенда с использованием EF6, Firebase, CQRS, MediaTek и DDD. Он обрабатывал обмен данными в реальном времени, безопасные платежи через Stripe и сложные потоки данных. Архитектура стала основой для будущих проектов и значительно улучшила удобство сопровождения.

9. Как вы обеспечиваете качество кода в своих проектах?

Я придерживаюсь принципов чистого кодирования, провожу последовательные проверки кода и активно использую модульное тестирование с помощью xUnit, NUnit и Moq. Также я использую такие инструменты, как Swashbuckle, для документирования API и конвейеры CI/CD для раннего выявления проблем. Я считаю, что качество — это не конечный этап, а непрерывный процесс.

10. Что вы делаете, если не согласны с техническим решением?

Я привожу к обсуждению данные, примеры и потенциальные риски. Я стараюсь понять логику решения и предлагаю альтернативы, если это необходимо. Моя цель — не «победить», а найти лучшее решение для проекта. Если команда принимает иное решение, я поддерживаю его и помогаю эффективно его реализовать.

11. Как вы справляетесь со сжатыми сроками?

Я разбиваю задачи на более мелкие этапы, расставляю приоритеты для критически важных функций и четко сообщаю о том, что реально может быть реализовано. Я фокусируюсь на стабильности и избегаю упрощений, которые могут привести к техническому долгу. При необходимости я предлагаю поэтапную реализацию для обеспечения качества.

12. Каков ваш подход к изучению новых технологий?

Я начинаю с официальной документации, создаю небольшие прототипы, а затем применяю технологии к реальным задачам проекта. Я также слежу за лидерами отрасли, читаю технические блоги и экспериментирую с такими инструментами, как помощники на основе GPT, чтобы ускорить обучение.

13. Расскажите о случае, когда вы улучшили командные процессы.

В DARS я внедрил методы структурированного модульного тестирования и помог команде освоить конвейеры CI/CD в Azure DevOps. Это уменьшило количество ошибок при развертывании и повысило согласованность релизов. Я также задокументировал лучшие практики использования CQRS и MediatR, что помогло стандартизировать кодовую базу.

14. Что вам больше всего нравится в разработке бэкенда?

Мне нравится проектировать системы, которые являются быстрыми, надежными и масштабируемыми. Мне нравится решать сложные задачи обработки данных, оптимизировать производительность и строить архитектуры.

которая может расти вместе с бизнесом. Разработка бэкенда дает мне возможность сочетать логику, структуру и креативность.

15. В каком направлении вы видите свое дальнейшее развитие?

Я хочу продолжать совершенствовать свои архитектурные навыки, работать над крупными распределенными системами и в конечном итоге взять на себя больше руководящих обязанностей. Меня также интересует более глубокая интеграция ИИ, блокчейна и облачных технологий в корпоративные приложения.